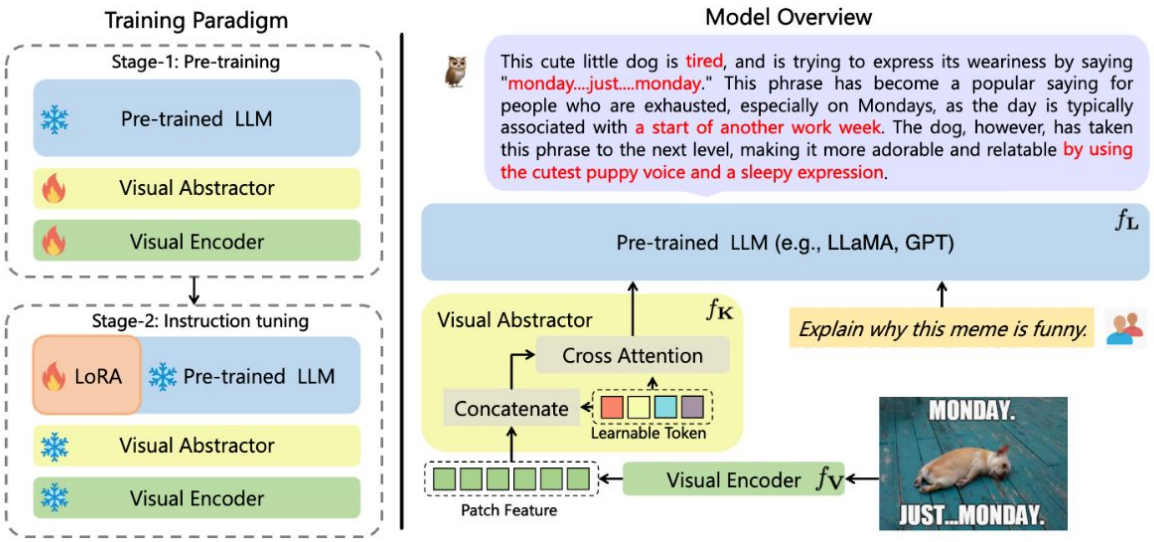
mPLUG-OWL(2023, Alibaba)

论文：《mPLUG-Owl: Modularization Empowers Large Language Models with Multimodality》
论文地址：https://arxiv.org/pdf/2304.14178
项目地址：https://github.com/X-PLUG/mPLUG-Owl

1. 模型结构



mPLUG-owl 已经和最新 (2025 年) 的 mllm 结构基本一致了，也是分为三个模块：

1.1 Visual Encoder

使用的是 ViT-L/14（num_layers = 24，hidden_dimension=1024，patch_size=14）。

参数量大约 0.2B。

1.2 LLM

LLaMA-7B pretrain 版本。

1.3 Adapter（Visual Abstractor）

这个结构参考的是Flamingo(2022, DeepMind)的 Resampler，虽然BLIP-2(2023, Salesforce Research)也使用类似 Learnable Token 的技术，但是有一个细节，mPLUG 和Flamingo(2022, DeepMind)的 Resampler 会讲 Learnable Token 拼接到视觉 KV 中去。

```
class MplugOwlVisualAbstractorAttention(nn.Module):
    def __init__(self, config: MplugOwlVisualAbstractorConfig):
        super().__init__()
        self.attention = MplugOwlVisualAbstractorMultiHeadAttention(config)
        self.output = MplugOwlVisualAbstractorCrossOutput(config)
        self.pruned_heads = set()
        self.norm1 = LayerNormFp32(config.hidden_size)
        self.normk = LayerNormFp32(config.hidden_size)

    def prune_heads(self, heads):
```

```
    if len(heads) == 0:
        return
    heads, index = find_pruneable_heads_and_indices(
        heads, self.attention.num_attention_heads, self.attention.attention_head_size, self.
pruned_heads
    )

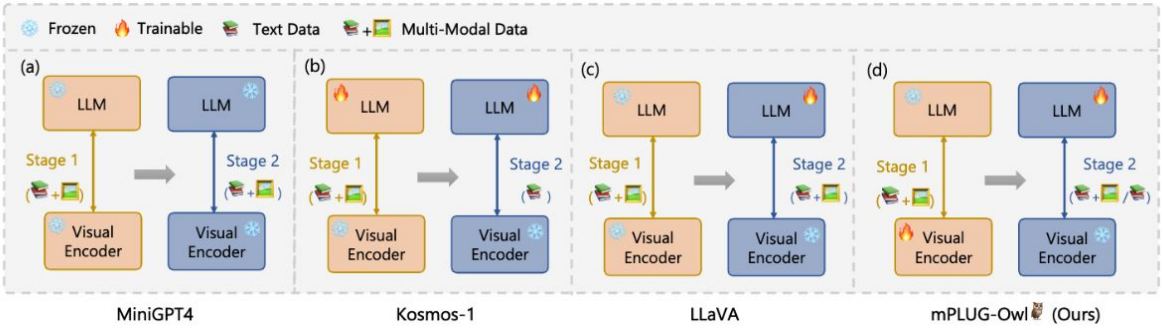
    # Prune linear layers
    self.attention.query = prune_linear_layer(self.attention.query, index)
    self.attention.key = prune_linear_layer(self.attention.key, index)
    self.attention.value = prune_linear_layer(self.attention.value, index)
    self.output.dense = prune_linear_layer(self.output.out_proj, index, dim=1)

    # Update hyper params and store pruned heads
    self.attention.num_attention_heads = self.attention.num_attention_heads - len(heads)
    self.attention.all_head_size = self.attention.attention_head_size * self.attention.
num_attention_heads
    self.pruned_heads = self.pruned_heads.union(heads)

def forward(
    self,
    hidden_states: torch.Tensor,
    attention_mask: Optional[torch.FloatTensor] = None,
    head_mask: Optional[torch.FloatTensor] = None,
    encoder_hidden_states: Optional[torch.FloatTensor] = None,
    encoder_attention_mask: Optional[torch.FloatTensor] = None,
    past_key_value: Optional[Tuple[Tuple[torch.FloatTensor]]] = None,
    output_attentions: Optional[bool] = False,
) -> Tuple[torch.Tensor]:
    # HACK we apply norm on q and k
    hidden_states = self.norm1(hidden_states)
    encoder_hidden_states = self.normk(encoder_hidden_states)
    encoder_hidden_states = torch.cat([hidden_states, encoder_hidden_states], dim=1)
    encoder_attention_mask = torch.cat([attention_mask, encoder_attention_mask], dim=-1)
    self_outputs = self.attention(
        hidden_states,
        attention_mask,
        head_mask,
        encoder_hidden_states,
        encoder_attention_mask,
        past_key_value,
        output_attentions,
    )
    attention_output = self.output(self_outputs[0], hidden_states)
    # add attentions if we output them
    outputs = (attention_output,) + self_outputs[1:]
    return outputs
```

2. 训练

下面这张图很好的展示了 mPLUG-OWL 和一些同期模型的训练区别。



2.1 Stage 1

使用的数据格式是 image-caption pairs，采样于 LAION-400M、COYO-700M、Conceptual Caption、MSCOCO。

使用了批量大小为 2.1 million 个 token，并将 mPLUG-Owl 训练 50k 步，对应大约 104 billion 个 token。我们采用 AdamW 优化器，β 参数设为 (0.9, 0.98)，学习率和权重衰减系数分别设置为 0.0001 和 0.1。训练初期先进行 2000 步的学习率预热（warm-up），然后采用余弦退火（cosine schedule）来衰减学习率。输入图像会被随机调整为 224 × 224 的尺寸。

文本编码器使用的是 SentencePiece。

2.2 Stage 2

在第二阶段，从三个不同来源收集了纯文本指令数据，102k data from the Alpaca、90k from the Vicuna 和 50k from the Baize。此外，我们还使用了来自 LLaVA 数据集的 50k 条多模态指令数据。

对 mPLUG-Owl 进行 2000 步训练，批量大小为 256，学习率设为 2*10^-5。